

PARKFLOW360

GUÍA 01

Estructuración del Frontend en WebStorm con Vite

Versión del Estudiante · Implementación guiada sin solucionario de actividades

Stack	React + TypeScript + Vite
IDE	JetBrains WebStorm
Proyecto	ParkFlow360
Versión de la guía	1.0 · 23/09/2026

Objetivo: construir una base frontend limpia, profesional y preparada para consumir el backend REST de Cliente y Vehículo.

Resultado esperado: al finalizar tendrás un proyecto React + TypeScript + Vite ejecutándose, ordenado por módulos y listo para comenzar la interfaz de ParkFlow360.

Contenido de la guía

1. Propósito y alcance
2. Mapa Fullstack de ParkFlow360
3. Requisitos previos
4. Creación del proyecto con Vite
5. Primera ejecución
6. Comprender la estructura generada
7. Limpieza del proyecto inicial
8. Estructura profesional propuesta
9. Creación de carpetas en WebStorm
10. Convenciones de nombres
11. Configuración inicial preparada para el backend
12. Práctica obligatoria
13. Preguntas de defensa oral
14. Lista de verificación y entrega

1. Propósito y alcance

En el backend de ParkFlow360 ya se trabajó con las entidades Cliente y Vehículo, una relación uno-a-muchos y operaciones REST probadas mediante Postman y Swagger. Esta guía inicia el frontend que posteriormente consumirá esos endpoints.

- Crear correctamente un proyecto con React, TypeScript y Vite desde WebStorm.
- Distinguir Node.js, npm, Vite, React y TypeScript.
- Comprender qué archivos crea Vite y para qué sirve cada uno.
- Organizar el código por responsabilidades y por funcionalidades del dominio.
- Dejar preparados los módulos de clientes y vehículos sin conectarlos todavía al backend.
- Ejecutar, detener y volver a levantar el servidor de desarrollo desde WebStorm.

Alcance de esta guía: estructura y arranque. No instalaremos todavía Axios, React Router, librerías de formularios ni componentes visuales externos.

2. Mapa Fullstack de ParkFlow360

La aplicación completa debe entenderse como una cadena de responsabilidades. El navegador no debe comunicarse directamente con PostgreSQL.

Visión general del flujo

```
NAVEGADOR
↓
REACT + TYPESCRIPT
↓
PÁGINAS / COMPONENTES / FEATURES
↓
SERVICIO HTTP DEL FRONTEND
↓
API REST PARKFLOW360 (SPRING BOOT)
↓
SERVICIOS / PUERTOS / ADAPTADORES
↓
JPA / HIBERNATE
↓
POSTGRESQL
```

En esta primera guía trabajaremos únicamente en la parte superior de ese flujo: creación, organización y ejecución del frontend.

3. Requisitos previos

3.1 Software

- ☐ WebStorm instalado y funcionando.
- ☐ Node.js compatible con la versión actual de Vite.
- ☐ npm disponible desde terminal.
- ☐ Plugins de React y Vite habilitados en WebStorm.

❑ Backend ParkFlow360 guardado por separado; no mezclar código Java y React en la misma carpeta src.

3.2 Verificación desde la terminal de WebStorm

```
node -v  
npm -v
```

Si ambos comandos muestran una versión, Node.js y npm están disponibles. Para Vite actual, usa Node.js 20.19+ o 22.12+ como mínimo.

3.3 Habilitar plugins en WebStorm

1. Abrir Settings con Ctrl+Alt+S.
2. Entrar a Plugins → Installed.
3. Buscar React y Vite.
4. Verificar que estén habilitados.
5. Reiniciar WebStorm si el IDE lo solicita.

4. Creación del proyecto con Vite

4.1 Opción recomendada para clase: terminal integrada

La terminal integrada permite que todos los estudiantes reproduzcan exactamente los mismos pasos, independientemente de pequeñas diferencias en la interfaz gráfica del IDE.

```
npm create vite@latest parkflow360-frontend -- --template react-ts  
cd parkflow360-frontend  
npm install
```

No escribas espacios ni tildes en el nombre físico del proyecto. Usa minúsculas y guiones: parkflow360-frontend.

4.2 Opción gráfica desde WebStorm

1. File → New → Project.
2. Seleccionar React o Vite, según la pantalla disponible en tu versión de WebStorm.
3. Seleccionar un Node runtime válido.
4. Elegir create-vite como generador.
5. Elegir React como framework y TypeScript como plantilla.
6. Definir la carpeta parkflow360-frontend y crear el proyecto.

En ambos casos el resultado conceptual debe ser el mismo: un proyecto generado por create-vite usando la plantilla react-ts.

5. Primera ejecución

```
npm run dev
```

Vite mostrará una dirección local. Ábrela desde el enlace de la terminal o desde el navegador. Debes observar la aplicación inicial de React/Vite.

- ☐ El servidor inicia sin errores.
- ☐ La URL local abre en el navegador.
- ☐ Al modificar un texto de App.tsx y guardar, el navegador se actualiza.
- ☐ Puedes detener el servidor con Ctrl+C.
- ☐ Puedes volver a iniciarlo con npm run dev.

5.1 Verificación adicional

```
npm run build
```

La compilación de producción debe finalizar sin errores. Vite generará la carpeta dist/, que no se debe editar manualmente.

6. Comprender la estructura generada

Elemento	Función
node_modules/	Dependencias instaladas por npm. No se modifica manualmente ni se sube a Git.
public/	Archivos públicos estáticos que pueden copiarse sin transformación.
src/	Código fuente principal de la aplicación.
src/main.tsx	Punto de entrada de React; monta la aplicación en el DOM.
src/App.tsx	Componente raíz generado por Vite; será reorganizado.
index.html	Documento HTML base que contiene el nodo donde React se monta.
package.json	Describe el proyecto, scripts y dependencias.
package-lock.json	Registra versiones resueltas de dependencias para instalaciones reproducibles.
tsconfig*.json	Configuración del compilador y del análisis de TypeScript.
vite.config.ts	Configuración de Vite y plugins.
eslint.config.*	Reglas de análisis estático y calidad de código.

Importante: node_modules no es “el código del proyecto”. Es material instalado a partir de package.json y package-lock.json.

7. Limpieza del proyecto inicial

Antes de construir ParkFlow360 elimina solamente el contenido de demostración que trae la plantilla. No elimines archivos de configuración sin comprenderlos.

1. Revisar App.tsx y reconocer el contenido de demostración.
2. Eliminar logos e imágenes de ejemplo que no se usarán.
3. Simplificar el CSS inicial si contiene estilos exclusivos de la demo.
4. Mantener main.tsx como punto de entrada.
5. Verificar nuevamente con `npm run dev` y `npm run build`.

```
// Objetivo mínimo temporal de App.tsx
export default function App() {
  return (
    <main>
      <h1>ParkFlow360</h1>
      <p>Frontend en construcción</p>
    </main>
  );
}
```

8. Estructura profesional propuesta

Usaremos una estructura orientada a funcionalidades (feature-based) y responsabilidades. Esto evita tener cientos de archivos mezclados cuando aparezcan más dominios del sistema.

Estructura base para ParkFlow360

```

src/
├── app/
│   └── App.tsx
├── assets/
├── components/
│   └── common/
├── config/
├── features/
│   ├── clientes/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── services/
│   │   └── types/
│   └── vehiculos/
│       ├── components/
│       ├── pages/
│       ├── services/
│       └── types/
├── hooks/
├── layouts/
├── pages/
├── routes/
├── services/
│   └── http/
├── styles/
├── types/
├── utils/
└── main.tsx

```

Carpeta	Responsabilidad
app/	Composición principal de la aplicación.
assets/	Imágenes, iconos y recursos importados desde el código.
components/common/	Componentes verdaderamente reutilizables: botones, inputs, tablas, mensajes.
config/	Configuraciones generales y lectura controlada de variables de entorno.
features/clientes/	Todo lo específico del módulo Cliente.
features/vehiculos/	Todo lo específico del módulo Vehículo.
hooks/	Hooks reutilizables que no pertenecen a un único módulo.
layouts/	Estructuras de pantalla: menú lateral, cabecera, contenido, etc.
pages/	Páginas globales como Inicio, 404, Acerca de.

Carpeta	Responsabilidad
routes/	Definición central de rutas cuando se incorpore React Router.
services/http/	Cliente HTTP y configuración común para comunicarse con Spring Boot.
styles/	Estilos globales, variables y temas.
types/	Tipos TypeScript globales o compartidos.
utils/	Funciones utilitarias puras y reutilizables.

9. Creación de carpetas en WebStorm

1. En el panel Project, expandir src.
2. Clic derecho sobre src → New → Directory.
3. Crear una carpeta por vez: app, assets, components, config, features, hooks, layouts, pages, routes, services, styles, types y utils.
4. Dentro de components crear common.
5. Dentro de services crear http.
6. Dentro de features crear clientes y vehiculos.
7. Dentro de cada feature crear components, pages, services y types.
8. Mover App.tsx a src/app/App.tsx usando Refactor → Move cuando sea posible.
9. Actualizar el import correspondiente en main.tsx.
10. Ejecutar npm run dev y corregir cualquier import roto.

En frontend, “crear paquete” normalmente significa organizar módulos/carpetas. No es equivalente al package de Java. Un paquete npm es otra cosa: una dependencia instalada con npm.

10. Convenciones de nombres

Tipo	Convención	Ejemplo
Componentes React	PascalCase	ClienteForm.tsx
Páginas React	PascalCase + Page	CientesPage.tsx
Hooks	camelCase comenzando con use	useClientes.ts
Servicios	camelCase	clienteService.ts

Tipo	Convención	Ejemplo
Tipos/Interfaces	PascalCase	Cliente.ts
Utilidades	camelCase	formatDate.ts
Carpetas de feature	minúsculas	clientes/
Variables/funciones	camelCase	obtenerClientes

11. Configuración inicial preparada para el backend

Todavía no consumiremos la API, pero dejaremos previsto dónde vivirá la URL del backend. Vite expone al código cliente solamente variables de entorno cuyo nombre inicia con `VITE_`.

Contenido futuro de `.env.local` (ajusta el puerto al backend real)

```
VITE_API_BASE_URL=http://localhost:8080
```

Nunca guardes contraseñas, tokens privados ni secretos del servidor en variables `VITE_`. El código frontend termina en el navegador del usuario.

Por ahora basta con crear un archivo `.env.example` con el nombre de la variable y una URL de ejemplo, sin credenciales.

12. Práctica obligatoria

Realiza las siguientes actividades de manera individual. La guía no incluye el solucionario de estas tareas.

- ☐ Crear el proyecto `parkflow360-frontend` con la plantilla `react-ts`.
- ☐ Ejecutar el proyecto y capturar evidencia de la aplicación funcionando.
- ☐ Reorganizar `App.tsx` dentro de `src/app` y actualizar los imports necesarios.
- ☐ Crear toda la estructura de carpetas definida en la sección 8.
- ☐ Crear un archivo `README-front.md` explicando con tus propias palabras el propósito de cinco carpetas.
- ☐ Crear un componente vacío `ClienteCard.tsx` dentro de `features/clientes/components`.
- ☐ Crear un componente vacío `VehiculoCard.tsx` dentro de `features/vehiculos/components`.
- ☐ Crear un tipo `Cliente.ts` dentro de `features/clientes/types` y un tipo `Vehiculo.ts` dentro de `features/vehiculos/types`, sin copiar todavía entidades Java de forma automática.
- ☐ Ejecutar `npm run build` y registrar cualquier error que aparezca.
- ☐ Cerrar WebStorm, abrir nuevamente el proyecto y demostrar que puedes levantarlo sin repetir `npm create vite`.

13. Preguntas de defensa oral

1. ¿Qué diferencia existe entre Node.js, npm, Vite, React y TypeScript?
2. ¿Qué problema resuelve Vite durante el desarrollo?
3. ¿Qué archivo es el punto de entrada de React en este proyecto?
4. ¿Para qué sirve package.json?
5. ¿Por qué node_modules no debe subirse normalmente al repositorio?
6. ¿Qué ocurre cuando ejecutas npm run dev?
7. ¿Qué ocurre cuando ejecutas npm run build?
8. ¿Por qué Cliente y Vehículo se ubican en features diferentes?
9. ¿Cuál es la diferencia entre components/common y features/clientes/components?
10. ¿Por qué el frontend no debe conectarse directamente a PostgreSQL?
11. ¿Por qué no deberíamos guardar una contraseña en VITE_API_BASE_URL o en otra variable VITE_?
12. ¿Qué ventaja aporta TypeScript antes de ejecutar la aplicación?
13. Si mueves App.tsx y la aplicación deja de compilar, ¿qué revisarías primero?
14. ¿Qué significa HMR y cómo lo observaste durante la práctica?
15. ¿Qué parte del backend monolito hexagonal será invocada desde este frontend: la base de datos o la API HTTP? Justifica.

14. Lista de verificación y entrega

- ☐ El proyecto abre correctamente en WebStorm.
- ☐ npm install finaliza sin errores.
- ☐ npm run dev inicia la aplicación.
- ☐ npm run build finaliza correctamente.
- ☐ La estructura de carpetas coincide con la definida en esta guía.
- ☐ App.tsx está ubicado en src/app/.
- ☐ main.tsx importa correctamente App.
- ☐ No se sube node_modules/ al repositorio.
- ☐ Se entrega captura del navegador y captura del árbol de carpetas.
- ☐ El estudiante puede responder oralmente las preguntas de la sección 13.

Evidencias sugeridas: captura del árbol del proyecto, salida de npm run dev, salida de npm run build y URL local funcionando.

Siguiente etapa: instalar y estructurar React Router para construir el layout principal, el menú de navegación y las rutas de Clientes y Vehículos.

Referencias técnicas oficiales

Documentación verificada para esta guía (consulta realizada el 23/09/2026):

- [Vite – Getting Started](#)
- [JetBrains WebStorm – React](#)
- [JetBrains WebStorm – Vite](#)
- [React – Using TypeScript](#)

Nota de compatibilidad actual: Vite requiere Node.js 20.19+ o 22.12+; si el generador o npm muestra un requisito superior, se debe actualizar Node.js antes de continuar.